



Python Pocket Reference

bxuleitario de funciones más utilizadas

Contenido

Python Pocket Reference.....	1
Contenido	1
Python	1
Paradigmas de programación.....	1
Entornos Virtuales .venv	1
Data Types	1
Operadores.....	2
Estructuras de Control.....	3
Funciones	3
Ficheros	3
Numpy	4
Pandas.....	4
Matplotlib.....	6

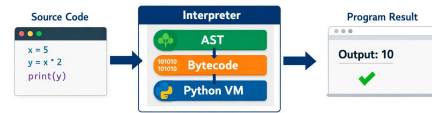
Python

Características:

- **Lenguaje** de propósito general
- Lenguaje de alto nivel
- **Interpretado**, no compilado: más portable y flexible.
- **Tipado dinámico**: variables de tipo dinámico
- **Fuertemente tipado**: operaciones str con int o float **ERROR**.
- Gestión automática de memoria: performance ↓
- Multiparadigma: POO, Procedimental, Estructurado.
- Multiplataforma
- **Librerías**: infinidad de librerías para múltiples tareas.

Ejecución:

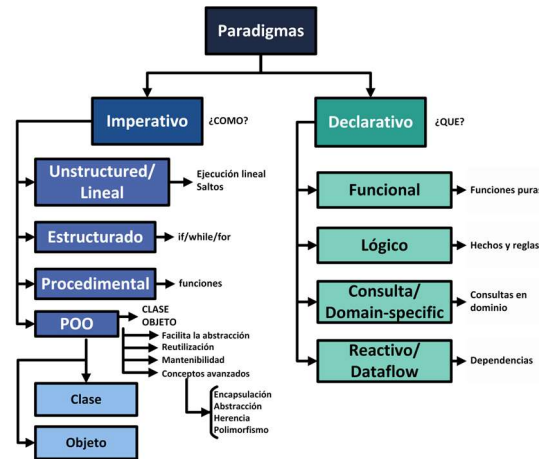
- El código fuente se tokeniza y comprueba errores
- Se genera Abstract Syntactic Tree AST. Representación estructurada del programa para optimizaciones.
- Convertir a bytecode: Código intermedio que describe mediante bytes las operaciones. Fichero .pyc.
- PVM: Python Virtual Machine se encarga de leer bytecode y ejecutarlo



Paradigmas de programación

Python es multiparadigma y está basado en Objet Oriented. Los paradigmas se dividen en dos grandes ramas:

- **Imperativo**: La programación define la tarea completa, con todos los pasos necesarios para conseguir el objetivo.
- **Declarativo**: Se declaran objetivos finales y es el programa el encargado de buscar la manera de conseguirlo.



Entornos Virtuales .venv

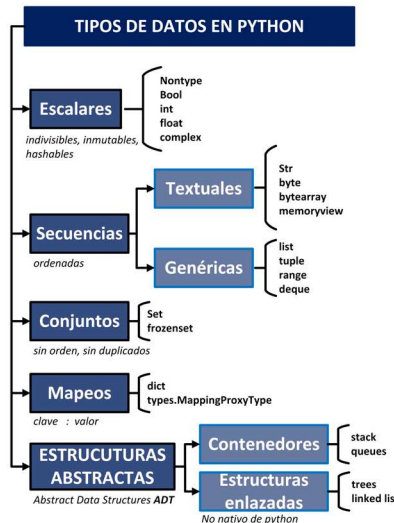
Los entornos virtuales son congelaciones de versiones, almacenadas en un directorio que permiten evitar problemas de compatibilidad entre librerías y versiones de Python. Se pueden crear con [anaconda](#) o [Python](#).

```
• conda: "anaconda prompt"
  directorio
cd C:\\"Directorio deseado"
  crear
conda create --prefix ..conda_env python=3.11 -y
  activar
conda activate ..conda_env
  instalar jupyter kernel
conda install ipykernel -y
  declarar con nombre: e. g. deep_learning
python -m ipykernel install --user --name
"deep_learning" --display-name "MIAR (deep_learning)"
  Instalar librerías necesarias con pip o condas
pip install seaborn
conda install numpy pandas matplotlib -y
  Desactivar entorno virtual
Conda deactivate
```

```
• python: cmd. Instala versión de Python actual del sistema.
  directorio
cd C:\\"Directorio deseado"
  crear
py -m venv .venv
  activar
.\.venv\Scripts\activate
  instalar jupyter kernel
pip install ipykernel
  declarar con nombre: e. g. deep_learning
python -m ipykernel install --user --name deep_learning
--display-name "MIAR (deep_learning)"
  Instalar librerías necesarias con pip
pip install numpy pandas matplotlib seaborn
```

```
• Para guardar instalaciones necesarias y recuperarlas.
Entorno virtual activo
Para guardar librerías
pip freeze > requirements.txt
Para instalar todo en el entorno virtual activo.
pip install -r requirements.txt
```

Data Types



Bool

- True / False
- Se guardan en caché. Mismo id para distintas variables iguales.

```
a = False
b = True
c = False
print(f"a= {a}, b= {b}")
print(f"type a= {type(a)}, type b= {type(b)}")
print(f"id_a={id(a)}\nid_c= {id(c)}\na is c={a
is c}")
```

Números

```
• Tipos:
2      # Enteros, sin parte fraccional.
3.4    # Números en coma flotante
2+4j   # Números complejos.
1/2    # Números racionales.

• Representaciones en diferentes bases:
x = 58      # decimal
z = 0b111010 # binario
w = 0o72    # octal
y = 0x3A    # hexadecimal
binario = bin(x) # binario formato texto
octal = oct(x)  # octal formato texto
hexad = hex(x)  # hexadecimal formato texto

x == y == z == w
int(binario, 0) == int(octal, 0)

complex(4, 3)
```

String

```
• Cadenas de caracteres.
• Son secuencias con posición.
• Inmutables. No cambian

s = 'John "ee" Doe'
s[1] # Primer carácter del string.
s[-1] # Último carácter del string.
s[3:] # Substring desde el segundo carácter
(inclusive) hasta el octavo (exclusive). Esta
técnica se la conoce como 'slicing'.
s[:] # Todo el string.
s + "e" # Concatenación.
(s + "e")[::-1] # Invertir un string
```

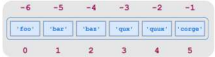
• Métodos propios:	
Trim	strip/lstrip/rstrip
Case	lower/upper/title/capitalize/swapcase/casefold
Buscar	startswith/endswith/find/count
Split	split/rsplit/splitlines/join
Checks	isupper/islower/istitle/isalpha/isalnum/isdigit/isdecimal/isnumeric/isspace/isidentifier

Conversiones (casting)

```
a = int(2.8)      # a será 2
b = int("3")      # b será 3
x = int("0b10101", 0) # base 0 = texto
c = float(1)      # c será 1.0
d = float("3")    # d será 3.0
e = str(2)        # e será '2'
f = str(3.0)      # f será '3.0'
g = bool("a")     # g será True
h = bool("")      # h será False
i = bool(3)       # i será True
j = bool(0)       # j será False
k = bool(None)    # None será False
```

Listas

- Colección de objetos heterogéneos (int, float, str, list...).
- Con duplicados.
- Sin tamaño fijo.
- Orden y acceso por índice: [índice].
- Tamaño variable.
- Soportan anidamiento



```
# Declaración
letras = ['a', 'b', 'c', 'd']
palabras = 'Hola mundo'.split()
numeros = list(range(5))
vacía = list() # []
mezcla = [1, 3.4, 'a', None, False]

# Incluso objetos más "complejos"
lista_con_funcion = [1, 2, len] # funciones
built-in o propias
lista_con_funcion[2](range(200))
```

- Slicing [inicio : fin : paso]

```
letras[1:3] # de 1 a 2
letras[:1] # de 0 a 0
letras[:-1] # de 0 a penúltima
letras[2:] # de 2 a final
letras[:] # completa
letras[1::2] # 1 a final cada 2
letras[::-1] # invertir
```

- Copy / Deep copy

```
# Shallow copy
# copia de lista (diferentes id interno).
# id ANIDADOS IGUALES!!
copia = letras.copy()
copia = letras[:]

# deep copy
import copy
copia = copy.deepcopy(lista)
```

- Funciones y operaciones

```
# Añade al final
letras.append('e')
# Inserta en posición
letras.insert(1,'g')
# devuelve el valor y elimina
letras.pop()
letras.index('b') # índice first "b" or ERROR
letras.remove('a')# elimina first "a" or ERROR

# Concatenar listas
lacteos = ['queso', 'leche']
frutas = ['naranja', 'manzana']
compra = lacteos + frutas
# ['queso',..., 'manzana']

frutas.extend(['pera', 'uvas'])# Extender list

lacteos_3 = lacteos * 2 # repite n veces list

# Ordenar. default Reverse= False
lista.sort(reverse=True) # solo homogéneos
```

Diccionarios

- Colección de {clave : valor}.
- Mutables.
- Claves:
 - Cualquier objeto inmutable.
 - Únicas.
- Valores:
 - Sin restricciones.
 - Permite anidamiento.

- Creación

```
# Creación
# Simple, usando una expresión literal.
persona = {'DNI': '1234N', 'Nombre': 'Madariaga'}
# Uniendo dos colecciones. len =
nombres = ['Sistiaga', 'Sabino']
edades = [52, 14]
datos = dict(zip(nombres,edades))
# Pasando claves y valores a la función 'dict'
persona2 = dict(nombre='Idoia',
apellido='Riaño')
# Usando una lista de tuplas de dos elementos.
persona2 = dict([('nombre', 'Idoia'),
('apellido','Riaño')])
# Incremental por medio de asignación
persona = {}
persona['DNI'] = '11111111D'
persona['Nombre'] = 'Madariaga'
persona['Edad'] = 34
```

- Acceso y Modificación por key.

```
# Acceso
# Claves inexistentes produce error
print(persona['Nombre'])
# Clave, sino por defecto e. g. 180
persona.get('Estatura', 180)
# Modificación
persona['Nombre'] = 'Nicolás'# Cambiar value
persona['Edad'] += 1
del persona['Ciudad'] # Eliminar clave:valor
value = persona.pop('Edad') # return & del
# Comprobación de existencia de clave
'Nombre' in persona
```

- Métodos propios de diccionarios.

```
persona.items() # Devuelve todas (key, value)
persona.keys() # Devuelve todas las key
persona.values() # Devuelve todos los value
```

```
# iterar sobre (key, value)
for key, value in dict_personas.items():
    print('Clave: '+str(key)+' ,valor: '+ str(value))
```

Set (Conjuntos)

- Colecciones de elementos heterogéneos.
- Mutables.
- Sin duplicados.
- Sin orden, ni anidamiento.
- Solo contienen inmutables.

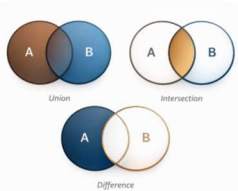
- Creación.

```
set1 = {0, 1, 1, 2, 3, 4, 4}
set2 = {'user1', 12, 2}
set3 = set(range(7))
set4 = set([0, 1, 2, 3, 4, 0, 1])
set5 = set() # Set vacío. No sirve set = {}
```

- Operaciones .

```
set1 = {0, 1, 1, 2, 3, 4, 5, 8, 13, 21}
set2 = set([0, 1, 2, 3, 4, 42])
# union
set1 | set2
# intersección
set1 & set2
# diferencia
set1 - set2
set2 - set1
```

```
# operaciones entre set y otro elementos
set1.union(lista) set1.intersection(lista)
set1.difference(texto)
```



- Comparación.

```
set2.issubset(set1)
set1.issuperset(set2)
set1.isdisjoint(set2)
```

- Modificación. Operación y asignación.

```
set1 |= set2 # set1 = set1 | set2
set1 &= set2
set1 -= set2
# Update admite cualquier dato (string se trocea)
set1.update(["1", 2, 7.4], "murcielago", set2)
```

- Anidamiento. Con tuplas

```
nested_set = {1, (1, 1, 1), 2, 3, (1,1,1)}
```

Tuplas

- Colección de elementos heterogéneos.
- Puede contener duplicados.
- Tiene orden y se accede a índices [índex] de [0 : n - 1]
- Soportan anidamiento.
- Inmutables.

- Creación

```
tuple1 = ('Foo', 34, 5.0, 34)
tuple2 = 1, 2, 3
tuple3 = tuple(range(10))
tuple4 = tuple([0, 1, 2, 3, 4])
# Ojo con tuplas de 1 elemento
singleton_number = (1) # Operador. NO ES TUPLA
singleton_number = (1,) # Tupla de un elemento
```

- Acceso por índice

```
tuple1[0] # primer elemento
tuple1[-1] # último
tuple1[-len(tuple1)] # primer elemento
tuple1[1:3] # de 1 a 2
```

- Métodos

```
tuple1.count(34) # N° ocurrencias
tuple1.index(34) # 1º posición | error

# Desempaquetar. *_ convierte a lista el restante
a, b, *_ = tuple1 # 'Foo' 34 [5.0, 34]
```

Secuencias

- Objetos iterables no materializados

- range(inicio, final, paso) (final no incluido)

```
secuencia = range(0,10,2)
lista = [secuencia]
tuple(secuencia)
set(secuencia)
```

- enumerate(iterable) → devuelve (índice, valor)

```
name = ['Urrusolo', 'Xalbador', 'Peixoto']
for ind, val in enumerate(name):
    print('Ind =' +str(ind)+' .Val =' + str(val))

for ind, val in enumerate(range(0,10,2)):
    print('Ind =' +str(index)+' .Val =' + str(val))
```

- zip / unzip → unir / separar dos iterables

```
for name,age in zip(lista,range(30,42,3)):
    print(f'Name:{name},age:{str(age)}')
```

```
players = zip(name,range(30,42,3),[120,130, 140])
ns, es, al = zip(*jugadores) # unzip
```

Operadores

- Aritméticos.

Símbolo	Operación	Sintaxis
+	Suma	a + b
-	Resta	a - b
/	División	a / b
//	División Entera	a // b
%	Modulo Resto	a % b
*	Multiplicación	a * b
**	Exponenciación	a ** b

- Comparación.

Símbolo	Operación	Sintaxis
>	Mayor	a > b
<	Menor	a < b
==	Igual	a == b
!=	Distinto	a != b
>=	Mayor igual	a >= b
<=	Menor igual	a <= b

- Operadores lógicos.

Símbolo	Operación	Sintaxis
and	AND	a and b
or	OR	a or b
^	XOR (solo uno True)	a ^ b
not	NOT	Not a

- Bitwise.

Símbolo	Operación binaria	Sintaxis
&	AND	a & b
	OR	a b
^	XOR	a ^ b
~	NOT	~ a
>>	Desplazamiento derecha	a >> n
<<	Desplazamiento izquierda	a << n

- Operador y asignación.

Símbolo	Operación y asignación	Sintaxis
+=	Suma	a += b
-=	Resta	a -= b
/=	División	a /= b
//=	División Entera	a //= b
%=	Modulo Resto	a %= b
*=	Multiplicación	a *= b
**=	Exponenciación	a **= b
&=	AND	a & b
=	OR	a b
^=	XOR	a ^ b
>>=	Desplazamiento derecha	a >> n
<<=	Desplazamiento izquierda	a << n

- Operadores de Identidad

Operador	Resultado
a is b	True, si hacen referencia al mismo objeto
a is not b	True, si ambos no referencian al mismo objeto

- Operadores de Pertenencia.

Operador	Resultado
a in b	True, si a se encuentra en b
a not in b	True, si a no se encuentra en b

Estructuras de Control

If

- Ejecución condicional.
- Las condiciones pueden ser concatenadas de **or** , **and** y **not**.

```
if condicion1:      #
    print("1")
elif condicion2:    # opcional
    print("2")
else:               # opcional
    print("0")
```

- Ternario.

```
resultado = x if x >= 0 else -x
```

Switch

- Comparar valor de una variable con varios casos.
- varios casos mismo resultado, combinación con |
- Caso por defecto (sí no cumple ninguna condición con _)
- Control "flojo". Se detiene con el primer match.
 - El orden es IMPORTANTE.
- La variable a comparar puede ser compleja (list, dict, set...).

```
match a:
    case 'A':
        print("Astelehena")
    case 'L' | 'I':
        print("Asteburua")
    case _:
        print("Egun okerra")
```

- "guard" expresión con if

```
match a:
    case val if val > 0:
        print(f'{val} is a positive number')
    case val if val == 0:
        print(f'{val} is a zero')
    case _ if a < 0:
        print('negative')
```

While

- Bucle de ejecución continua:
 - Mientras se cumpla la condición.
 - O hasta llegar a un break.
- Se puede anidar.

```
while i < 100:
    i += 1
    if i % 10 == 0:
        continue # opcional
    if i >= 99:
        break    # opcional
    print(i)
```

For

- Permite recorrer secuencia u objeto iterable.
- continue salta una iteración.
- break detiene la iteración.
- Se puede anidar.

```
peliculas=['Aupa Etxebeeste','Urte berri on,Amona]
for película in películas:
    print(película)
for i in range(10):
    print(i)
```

- else se ejecuta solo si sale del bucle sin break

```
for i, val in enumerate(películas):
    print(f"pos {i}, peli {val}")
else:
    print("fin normal")
```

- break continue opcionales.

```
for i, element in enumerate(lista):
    if element == element_to_find:
        found = True
        break
    if i % 2 == 0:
        continue
    print(f"i: {i}, elemto: {element}")
```

List Comprehensions

- Expresión compacta para creación de secuencias.
- Tienen mejor rendimiento.
- Soportan anidamiento.

```
# Simple
[char * 2 for char in 'Iruñea']
# Extendido
[x for x in range(9) if x % 2 == 0]
# Completo
[x**2 if x>2 else x for x in range(-4,5) if x>0]
# También para diccionarios y set
{k:v for k,v in zip(items,price)}
{x for x in range(10)}
```

Excepciones

- Eventos que saltan en situaciones excepcionales.
- Alteran el flujo de ejecución.
- Python lanza excepciones automáticas a partir de errores.
- Se pueden programar.

- try / except / finally
 - El programa no se detiene y puede seguir ejecución normal.

```
lista = [6, 1, 0, 5]
i = 5
```

```
try:      # Se ejecuta si no da error
    a = lista[i]
except Exception as e: #Error.Captura e impresión
    print("Error = " + str(e))
finally:  # Se ejecuta siempre
    print('Bloque finally')
    b = lista[-1]
```

- raise: lanzar excepciones de manera explícita.

```
try:
    raise ValueError("boom") #declarada en except
    print("no llega")
except ValueError:
    print("capturada")
print("sigue")
```

Funciones

- Agrupar conjuntos de instrucciones.
- Se pueden llamar infinitas veces.
- Por defecto sus variables son locales.
- Pueden recibir valores de entrada y devolver una salida
- Soporta anidamiento

- Definición:

```
def clean_strings(strings : str, ops) -> list:
    '''
    Definición de la función
    '''
    result = []
    for value in strings:
        for function in ops:
            value = function(value)
            result.append(value)
    return result
```

- Argumentos arbitrarios: se reciben como tupla

```
def saludar(*names):
    print(type(names))
    for name in names:
        print("Hola " + name)
```

- keywords arguments. kwargs (key, value).

```
def consulta_sql(**kwargs):
    print(kwargs)
    print(type(kwargs))

    sql = "SELECT * FROM bicis WHERE "
    args = [f"{key}={value}" for key, value in
kwargs.items()]
    sql += " AND ".join((args))
    return sql

print(consulta_sql(station='alameda', free=10))
```

Funciones Lambda

- Funciones sin nombre.
- Pueden aparecer en listas o como parámetros de función.

```
# como parámetro de una función
cities.sort(key = lambda x : len(x))
# almacenarla en variable
f = lambda x : len(x)
for s in lista:
    print(f(s))
# Lista de lambdas
pows = [lambda x: x ** 0, lambda x: x ** 1]
# dict de lambdas
{'add': lambda: x + y, 'sub': lambda: x - y}
```

Recursividad

- Funciones que se llaman a si mismas:

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)
```

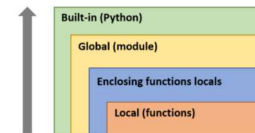
Scope

- La localización de asignación de variables determina acceso.
- En funciones, son internas

```
def func():
    def func_int():
        x = 30
        print("Valor en la función más interna:",
x)

        return
    x = 20 # Local a la función.
    func_int()
    print("Valor en la función intermedia:", x)
    return
x = 10
func()
print("Valor fuera de las funciones: ", x)
```

- Regla LEGB
- Local: dentro de funcion
- Enclosing: en funciones anidadas superiores
- Global: definidas en programa fuera de funciones
- Built-in: proporcionadas por el lenguaje Python.



Closure

- Las funciones pueden recordar su enclosing scope.

```
def f1():
    x = 88      # Enclosing scope (para f2)
    def f2():
        print(x)
    return f2   # Devuelve f2, sin invocarla

accion = f1()
accion()
```

- nonlocal modifica declaración de variables a no local (ni Global).

```
def f1():
    contador = 10
    def f2():
        nonlocal contador
        contador += 1
        print(contador)
    return f2
accion = f1()
accion()
```

Ficheros

- Built-in Python para manejar ficheros
- open(ruta, modo) crea objetos de tipo fichero
- fichero.readlines() lee las líneas del fichero
- with (opcional), gestiona apertura y cierre automático

```
import os # librería para directorios multisistem
#abrir varios ficheros en el mismo with
ruta1 = os.path.join("res", "one_line.txt")
ruta2 = os.path.join("res", "multiple_lines.txt")
with open(ruta1,"wt") as fichero1, open(ruta2) as
fichero2:
    print(fichero1.readlines())
    print(fichero2.readlines())
```

Modo	Descripción
r	Solo lectura
w	Solo Escritura (Borra si ya existe)
x	Solo Escritura (Falla si ya existe)
a	Crea Fichero (Si ya existe, open y añade al final)
r+	Lectura y Escritura
b	Se añade a otros para acceso binario
t	Acceso para archivos de texto (default)

CSV

- Lectura y escritura mediante librería

```
ruta = os.path.join("res",
"tabla_operaciones.csv")
with open(ruta, 'r+') as f:
    data_reader = csv.reader(f, delimiter = ',')
    csv_writer = csv.writer(f, delimiter = '|')
    for line in data_reader:
        print(line)
```

Docstring

- Se usan para escribir la documentación de objetos (funciones, métodos, módulos). Suelen tener:
 - Resumen en la primera línea.
 - Línea en blanco.
 - Descripción detallada.
 - Otra línea en blanco antes del código.

```
def funcion_de_prueba2():
    """Esta es la línea para el resumen.

    Descripción detallada
    """

    pass
```

- Se accede con help(). [Sphinx](#) y guía de estilo de [Google](#)

Numpy

- Numerical Python. Librería para análisis numérico.
 - Escrita en C, muy optimizada: ndarray
 - Colección de funciones para álgebra lineal y estadística descriptiva
- ```
import numpy as np
```

### Creación

```
my_arr = np.arange(10000000)#UtilizandoNumPy arr
array = np.array([[2,71,0], [2,171,-35]]) # 2D
shopping_list = [['onions','carrots'],
['apples','oranges']] # 2D
array = np.array(shopping_list) #lst to ndarray
```

Valores fijos.

```
Generar arrays con valores fijos
np.zeros(2) # 1D
np.ones((2,2),dtype = np.uint8)# matrix 2x2
np.empty((2,2,2)) # matrix 2x2x2
np.full((5,2,3) # matrix 5x2, todo 3s
np.eye(3) # matrix identity
```

### Información

- max y min

```
my_ndarray.argmax() # idx max value
my_ndarray.max() # max value
my_ndarray.argmin() # idx min value
my_ndarray.min() # min value
my_1darray.nonzero()# idx != 0
```

- Nº elementos, tamaño y forma

```
my_ndarray.size # N° of elements
my_ndarray.shape # dimension
my_ndarray.ndim # N° of dimensions
```

### Redimensionar

```
my_ndarray.reshape(2, 4) # cambia a 2x4
new_d = np.arange(9).reshape(3,3) # cambia a 3x3
new_d.flatten() # cambia a 1D
new_d.reshape(2,-1) # 2xi i se infiere
```

### Manipulación

```
new_dims.swapaxes(0,1) # cambiar ejes
np.flip(new_dims, axis = None) # invierte orden
array1.sort() # ordena
```

```
np.hstack((a1,a2)) # unión plana
np.vstack((a1,a2)) # unión apilada
```

```
np.concatenate((array1, array2), axis = 0) #
Concatena
```

```
np.array_split(array, X, axis = 1) # divide en X
partes iguales
np.array_split(array, [X], axis = 0) # divide,
por la fila X. admite [X,--,Z]
```

### Índices

- 1D

```
array[-1] # acceso 1D
array[:-1] # slice
```

- 2D

```
array[2, 2] # fila, columna
array[2][2] # fila, columna
```

- Multidimensional

```
array[1,2,4]
array[1][2][4]
```

### Row major

- Numpy es row major. Mayor por filas.
- Trabaja por filas

```
row_major = row_major.reshape(2,3,order='C') #
by default, row major
col_major.reshape(2,3,order='F') # col major
```

### Slice.

- Igual que en estructuras *[inicio : fin : paso]*
- Diferencia entre fila y columna *[fila, columna]*

```
array[1:3, 2::2]
array[0,:]
```

- Slice devuelve REFERENCIA. NO UNA COPIA

```
my_copy = array[2:5] # devuelve una referencia
```

- Copiar ndarray

```
my_copy = array[2:5].copy() # Copia
```

- Asignar sobre slice

```
array[0:3] = -1 # asigna -1 de 0 a 2
```

- Slicing condicionales. Devuelven copias

```
mask = array % 2 == 0 # [array de 800L]
array[mask] # [array mask]
array[array < 0] = 0
```

```
array_par = array[mask] # devuelve una copia
```

### Funciones matemáticas

- Escalar con array. Aplica a cada elemento **vectorial**.

```
array *= 2
array += 10
array /= 10
```

- Array con array. Aplica a cada elemento. Broadcasting

```
array_1 + array_2
array_1 - array_2
array_1 * array_2
array_1 / array_2
```

### Broadcasting

- Operación entre arrays de diferente tamaño.
- Si son longitud distinta pero formas coinciden.
- [broadcasting](#)

```
x = np.arange(6).reshape(2,3) # (2,3)
y = np.array([10,20,30]) # (3,)
x + y # OK: se alinea por la derecha
```

### Operadores unarios y binarios

- Unario. Un solo nd array. np.operator()

| Operación      | Descripción                                 |
|----------------|---------------------------------------------|
| abs            | Valor absoluto de cada elemento             |
| sqrt           | Raíz cuadrada de cada elemento              |
| exp            | e <sup>x</sup> . Siendo x cada elemento     |
| log, log10, .. | Logaritmos en distintas bases, a elementos  |
| sign           | Signo de cada elemento (-1, 0 o 1)          |
| ceil           | Redondea cada elemento por arriba           |
| floor          | Redondea cada elemento por abajo            |
| isnan          | Retorna si cada elemento es Nan             |
| cos, sin, tan  | Operaciones trigonométricas a cada elemento |
| Trig inv       | Operaciones trigonométricas inversas        |

- Binario. Entre dos array. np.operacion(). Broadcasting

| Operación                              | Descripción           |
|----------------------------------------|-----------------------|
| add                                    | Suma                  |
| subtract                               | Resta                 |
| multiply                               | Multiplicación        |
| divide                                 | divide                |
| maximum, minimum                       | Max y min de parejas  |
| equal, not_equal                       | Comparación de pareja |
| greater, greater_equal, les, les_equal | Diferencia de pareja  |

### Estadística descriptiva

| Función                 | Descripción                          |
|-------------------------|--------------------------------------|
| sum(arr)                | Suma de todos los elementos          |
| mean(arr)               | Media aritmética                     |
| std(arr)                | Desviación estandas                  |
| cumsum(arr)             | Suma acumulada                       |
| cumprod(arr)            | Producto acumulado                   |
| min(arr), max(arr)      | Mínimo y máximo                      |
| any(arr)                | Tipo BOOL. True si algún elemento es |
| all(arr)                | Tipo BOOL. TODOS.                    |
| unique(arr)             | Devuelve array con valores únicos    |
| in1d(arr1, arr2)        | Devuelve arr bool. arr1 en arr2      |
| union1d(arr1, arr2)     | Unión de ambos                       |
| intersect1d(arr1, arr2) | Intersección de ambos                |

### Álgebra lineal

| Función            | Descripción                 |
|--------------------|-----------------------------|
| dot(mat1, mat2)    | Producto escalar            |
| matmul(mat1, mat2) | Multiplicación matricial    |
| trace(mat1, mat2)  | Suma de diagonales de ambas |
| det(mat)           | Determinante de la matriz   |
| eig(mat)           | Autovalores y autovectores  |
| inv(mat)           | Inversa de la matriz        |
| qr(mat)            | Factorización QR de mat     |
| solve(A, b)        | Resuelve sistema lineal     |
| transpose(mat)     | Traspuesta de la matriz     |

### Filtrado de datos

- np.where permite filtrado de datos
- con retorno selectivo
- np.where(mask, si True, si Falso)*

```
np.where(prices < 1, 1.0, prices)#mask
np.where(np.isnan(data), 0, data)#filtrado de nan

selecció
credits = np.where(bank_transfers_values > 0)
```

### Números aleatorios

- Python

```
import random
random.random() # número entre 0 y 1
random.randint(0,5) # int entre dos valores
random.uniform(0,5) # real entre dos valores
```

- Numpy. Más distribuciones y admite size= (M x N).

```
np.random.rand() # número entre 0 y 1
np.random.randint(5, size=(2, 4)) # int de 2x4
```

```
np.random.randn(4,2) # Dist normal estandar
np.random.binomial(n=5, p=0.3) #binomial
np.random.uniform(low=0, high=10) # uniform
np.random.poisson(lam=2, size=(2,2)) # poisson
```

### Random seed

- En electrónica, número pseudoaleatorios.
- Ordenadores los generan a partir de ecuaciones.
- Basados en número inicial.
- Se toma seed nuevo cada vez para conseguir aleatoriedad “real”
  - o A partir del reloj (h:min:s.mili). Suficiente aleatoriedad
- Seed fijada, bueno para reproducibilidad de experimentos.

```
for _ in range(5):
 np.random.seed(i) # misma semilla
 print(np.random.rand())
 print(np.random.rand())
```

### Escritura y lectura ndarrays

- Formato \*.npy.
- Compartir datos.
- Copia de seguridad.

```
import os
ruta = os.path.join("res" , "values_array.npy")

values = np.random.randint(9, size=(3,3))
np.save(ruta, values) # Guardar. Permite multiple

old_values = np.load(ruta) # Leer archivo
```

- array.files. Contiene el nombre de los array.

```
npzfile = np.load(ruta)

print(type(npzfile.files))
print(npzfile.files)

for key in npzfile.files:
 print(npzfile[key])
```

- Bases de datos suelen estar en csv.
  - o más lentos de leer.
  - o myor uso de memoria.
  - o estándar.

```
csv es lento y pesado

ruta_csv = os.path.join("res" , "fdata.csv")
with open(ruta_csv, 'r') as f:
 data_str = f.read()
 data = data_str.split(',')
 data_array = np.array(data, dtype =
np.int8).reshape(1000,1000)

numpy es rápido y ligero
ruta_npy = os.path.join("res" , "o_fdata.npy")
np.save(ruta_npy, data_array)
```

## Pandas

```
import pandas as pd
```

### Series

- Datos 1D
- Elementos + índices modificables

```
Declaración
countries =
pd.Series(['Spain', 'Andorra', 'Gibraltar'],
index=["s", "a", "g"]) # idx str o int

Atributos
countries.name = 'Países' # name to series
countries.index.name = 'Id' # name to idx
Acceso
countries.loc['a', 's'] # índice semántico
countries.iloc[0, 2] # índice posicional
slicing
countries[:, 'g'] # incluye extremo
countries[:2] # No incluye extremo
```



- Cast

```
#cast a list
lista = list(countries.loc[:, 'g'])
Cast a ndarray
ndarray = np.array(countries.loc[:, 'g'])
cast a dictionary
dicc = dict(countries.loc[:, 'g'])
```

- Máscaras

```
fibonacci = pd.Series([0, 1, 1, 2, 3, 5, 8, 13])
mask = fibonacci > 10 # serie Bool
fibonacci[mask] # Series filtrado
```

- Reseteo de índices semánticos

```
fb.reset_index(drop=True, inplace=True)
```

- Basado en Numpy. Permite sus operaciones.

```
e. g. np.sum()
np.sum(fibonacci)
Filtrado con np.where
valid_distances =
np.where(pd.notnull(fibonacci), fibonacci, 0)
```

- Iteración

```
iterar sobre elementos
for value in fibonacci:
 print('Value: ' + str(value))

iterar sobre índices
for index in fibonacci.index:
 print('Index: ' + str(index))

iterar sobre elementos e índices
for index, value in fibonacci.items():
 a = 'Index: '+str(index) + ' Value: '+str(value)
```

- Series como diccionarios

```
crear key = idx : value = value
serie = pd.Series({Lertxundi : 100, 'Idoia':98})

añade y elimina elementos a través de índices
serie['Urrusolo'] = 12
del serie['Urrusolo ']

query una serie
if 'Lenrtxundi' in serie:
 l = serie['Lenrtxundi']
```

- Operaciones entre series.
  - Se opera por mismo índice.
  - Si no hay coincidencia NaN (np.nan).

```
serie1 = pd.Series([10,20,30,40], index=range(4))
serie2 = pd.Series([1,2,3], index=range(3))
suma = serie1 + serie2 # 11 22 33 NaN
```

## DataFrame

- Datos tabulares
- Columnas: Series con índices compartidos
- Columnas: Series con índices compartidos

```
diccionario = { "Nombre" :
["Marisa", "Laura", "Manuel"],
 "Edad" : [34,29,12] }

frame = pd.DataFrame(diccionario,
columns=['Nacionalidad', 'Nombre', 'Edad',
'Profesion'])
añadir columna
frame['Direccion'] = 'Desconocida'
```

- Modificación

```
Modificar una columna
lista_direcciones = ['Rue 13 del Percebe, 13',
'Evergreen Terrace, 3', 'Av de los Rombos, 12']
frame['Direccion'] = lista_direcciones

añadir fila (requiere todos los valores)
user_2 = ['Alemania', 'Klaus', 20, 'Ninguna',
'Desconocida']
frame.loc[3] = user_2

eliminar fila
frame.drop(2, axis= 0, inplace= True)
eliminar columna
frame.drop('Nombre', axis = 1, inplace = True)
```

- Iteración

```
iteración sobre columnas
for values in frame:
 values # Nombre de columna

iteración sobre filas
for values in frame.values:
 for value in values:
 value # valor de cada columna
```

- Indexación

```
axis= opcional por defecto 0
frame.iloc(axis= 1)[0] # Columna
frame.iloc(axis= 0)[0] # Fila
```

- Slicing. No modifican el DataFrame original.

```
frame.iloc[2, :] # [fila, columna]
frame.loc[:, ["Nombre", "Edad"]]
```

- Index. Inmutable

```
Construcción de índices
ind = pd.Index([20, 30, 50])
frame.index = ind

Cambiar columna index
frame.set_index('Edad', inplace=True)
```

- Mascaras

```
mask1 = frame['Edad'] < 10
mask2 = frame['Nombre'].str.contains("r")
frame.loc[~mask1 & mask2] # encadenar & | y not ~
```

- Cargar y guardar datos

```
Guardar a csv
import os
ruta = os.path.join("res" , "frame.csv")
```

```
frame.to_csv(ruta, sep=';') #sep por defecto: ','
```

```
Leer csv
loaded = pd.read_csv(ruta, sep=';')
```

## Búsqueda

- isin()

```
frame.isin(["Marisa", 34]) # DataFrame BOOL con
coincidencias
frame.isin(["Marisa", 34]).values # ndarray BOOL
frame.isin(["Marisa", 34]).values.sum() #
Contador ocurrencias
frame.isin(["Marisa", 34]).values.sum(axis= 0) #
Por eje
```

## Ordenación

```
ordenar por índice
frame.sort_index(ascending=True)

#ordenar por columna
frame.sort_index(axis=1, ascending=True, inplace=
True)

ordenar filas por valor en columna
frame.sort_values(by='Edad', ascending=True,
inplace= True)
```

## Rank

- Ranking. Para valores homogéneos (numéricos o string)
  - Ordena por valores ascendentes
  - Devuelve su posición en el ranking
  - Se puede hacer por filas o columnas el ranking
  - Tipos: 'average', 'min', 'max', 'first', 'dense']

```
frame.loc[:, ["Edad", "Peso"]].rank(method='max',
axis=1)
axis= 1 rank por fila
axis= 0 rank por columna
```

## Apply

- Aplica función al DataFrame
- Trabaja por columna o fila

```
rand_matrix = np.random.randint(10, size=(3, 4))
frame = pd.DataFrame(rand_matrix ,
columns=list('ABCD'))
Función a aplicar
def max_min(x):
 return x.max() - x.min()
Apply diferencia por columna
frame.apply(max_min, axis = 0) #
```

## Map

- Aplica función a cada valor

```
generación de prueba
rand_matrix = np.random.randint(10, size=(3, 4))
frame = pd.DataFrame(rand_matrix ,
columns=list('ABCD'))
frame["D"] = ["1a", "2a", "3a"]
```

```
función a aplicar
def value(x):
 if str(x).isdigit():
 if float(x) > 4:
 return float(x)
 else:
 return 0
 else:
 return np.nan
```

```
frame.map(value) # función a cada valor
```

## Estadística Descriptiva DataFrame.op()

| Operación      | Descripción                                    |
|----------------|------------------------------------------------|
| count          | Número de valores no NaN                       |
| describe       | Conjunto de estadísticas sumarias              |
| min, max       | Valores mínimo y máximo                        |
| argmin, argmax | Índices posicionales del valor mínimo y máximo |
| idxmin, idxmax | Índices semánticos del valor mínimo y máximo   |
| sum            | Suma de los elementos                          |
| mean           | Media de los elementos                         |
| median         | Mediana de los elementos                       |
| mad            | Desviación absoluta media del valor medio      |
| var            | Varianza de los elementos                      |
| std            | Desviación estándar de los elementos           |
| cumsum         | Suma acumulada de los elementos                |
| diff           | Diferencia aritmética de los elementos         |

## Agregaciones DataFrame.MultiIndex/grouby.agg()

- Permite agregar columnas multiples a partir de diccionario.
- Funciones definidas
  - sum, mean, median, min, max
  - count, size, nunique
  - std, var, sem
  - first, last, nth

```
df = pd.DataFrame({
 "grupo": ["A", "A", "B", "B"],
 "ventas": [10, 20, 30, 40],
 "items": [1, 2, 1, 3]})
```

|   | grupo | ventas | items |
|---|-------|--------|-------|
| 0 | A     | 10     | 1     |
| 1 | A     | 20     | 2     |
| 2 | B     | 30     | 1     |
| 3 | B     | 40     | 3     |

```
Agregar por grupo
res = df.groupby("grupo").agg(
 ventas_total=("ventas", "sum"),
 ventas_prom=("ventas", "mean"),
 items_total=("items", "sum"))
```

|       |    | vent_to | vent_prom | it_to |
|-------|----|---------|-----------|-------|
| grupo |    |         |           |       |
| A     | 30 | 15.0    | 3         |       |
| B     | 70 | 35.0    | 4         |       |

- Funciones propias y lambda

```
def rango(s):
 return s.max() - s.min()

res2 = df.groupby("grupo").agg({
 "ventas": [rango, lambda s: s.mean()],
 "items": "sum"})
```

- e. g.

```
diccionario = { "nombre" :
["Marisa", "Laura", "Manuel", "Carlos"], "edad" :
[34,34,11, 30],
 "puntos" : [98,12,98,np.nan],
"genero": ["F", "F", "M", "M"] }
frame = pd.DataFrame(diccionario)
funciones de agregación de varias columnas para
obtener distintos estadísticos
display(frame.groupby('genero')[['edad',
'puntos']].agg(['min', 'mean', 'max']))
```

## Unstack

- Aplica a MultiIndex o groupby

```
g = df.groupby(["grupo",
"items"])[["ventas"].sum()
display(g)
display(g.unstack("items"))
```

| grupo | items |    | items | 1    | 2    | 3    |
|-------|-------|----|-------|------|------|------|
| A     | 1     | 10 |       |      |      |      |
|       | 2     | 20 | grupo |      |      |      |
| B     | 1     | 30 | A     | 10.0 | 20.0 | NaN  |
|       | 3     | 40 | B     | 30.0 | NaN  | 40.0 |

Name: ventas, dtype: int64

## Correlaciones

- Entre columnas o entre todos

```
path = 'http://archive.ics.uci.edu/ml/machine-
learning-databases/auto-mpg/auto-mpg.data'
```

```
mpg_data = pd.read_csv(path, sep='\s+',
header=None,
names = ['mpg', 'cilindros',
'desplazamiento', 'potencia',
'peso', 'aceleracion', 'año',
'origen', 'nombre'],
na_values='?', engine='c')
Entre mpg y peso
mpg_data['mpg'].corr(mpg_data['peso']) # + mpg =
- peso
Entre todos, solo numérico
mpg_data.corr(numeric_only=True)
```